Problem Statement and Goals
Software Engineering

Team 19, Team BVSH
Shivya Mehta
Anindita Bornomala
Harmanpreet Singh Sagar
Vanessa Lai

Table 1: Revision History

Date	Developer(s)	Change
September 28, 2026	Shivya, Anindita, Harmanpreet, Vanessa	Initial version
...	...	...

1 Problem Statement

The Department of Computing and Software (CAS) lacks a centralized, coordinated method for managing course outline schedules and ensuring academic policy/engineering accreditation compliance, resulting in congested workload periods for students and high administrative overhead for staff and faculty.

1.1 Problem

Across the Department of Computing and Software (CAS), course outlines and assessment schedules are currently created and managed individually by instructors. As there is no centralized, coordinated system of determining when assignments and midterms are scheduled, students in the same program and academic year frequently face severe workload clustering. Having multiple midterms or major deliverables due within the same week or on consecutive days results in increased academic stress and poorer learning outcomes for students.

From an administrative perspective, coordinating all of these schedules and ensuring institutional guidelines are met is a time-consuming process. The CAS administrative staff must manually inspect course outlines to verify compliance

with departmental policies, university regulations, and Canadian Engineering Accreditation Board (CEAB) criteria. This process is further complicated by circumstances where missing or incomplete information is found in course outlines (e.g., midterms listed as "TBD"), resulting in administrative staff having to engage in back-and-forth communication with instructors to clarify details. Furthermore, as course instructors have full academic freedom over their course delivery, administrators are not authorized to reassign assignment deadlines or exam dates.

In summary, without a mechanism to identify scheduling bottlenecks and policy issues early, administrative workload remains high, staff and instructors are forced to spend time engaging in redundant communication, and students continue to suffer from congested work deadlines and midterm dates.

1.2 Inputs and Outputs

Inputs:

1. **Course Outlines:** The system will take detailed course outlines from the current academic year as input, including assignment deadlines, midterm dates, and course learning objectives. These outlines will be used to identify scheduling conflicts, evaluate workload distribution across courses, and assess compliance with CEAB accreditation requirements. For testing, the system will also take past or mock course outlines as input.

2. **Academic Calendar:** The system will take the current and future academic calendar year as input to take into consideration holidays, test and examination restrictions, and final examination periods, to ensure midterms, assignments and exams are scheduled correctly according to school policy.

Outputs:

1. **Shared Academic Calendar:** The system will provide a centralized, calendar-like interface displaying midterm dates and assignment deadlines across all undergraduate Software Engineering courses. Professors and administrators will be able to filter the calendar by course and academic year to view relevant deadlines and identify potential scheduling conflicts.

2. **CEAB Compliance Notifications:** The system will analyze course outlines against CEAB accreditation requirements and notify professors when their outlines do not meet the required criteria. These notifications will identify potential compliance issues and help professors address missing or incomplete information.

3. **Scheduling Recommendation Feature:** The system will provide professors with suggestions for adjusting midterm and assignment dates to reduce scheduling conflicts and student workload during particularly busy

weeks. Each recommendation will include an explanation of the reasoning behind the suggested changes, allowing professors to make informed scheduling decisions. Recommendations will be made instead of automated schedule changes, to ensure human verification.

1.3 Stakeholders

The key stakeholders for this project include:

- **CAS Administration:** The primary stakeholders and end-users of the system. They will use it to manage and maintain course information, monitor assessment schedules across CAS courses, and streamline the review of course outlines for CEAB accreditation requirements. The CAS Administration staff (Matt Vonk & Connie Carrabs) will also be responsible for overseeing the system's development and ensuring that it meets administrative needs.
- **Faculty Professors and Course Instructors:** End-users who would use the system to review assessment schedules across CAS courses and identify potential conflicts with other midterms and assignments. They will also be able to receive scheduling suggestions that propose alternative assessment dates and provide reasoning for how these changes could help reduce student workload and assessment conflicts.
- **Academic Advising Staff:** End-users who would use the centralized assessment schedule to better understand periods of high student workload and identify potential scheduling conflicts that could affect students. This information can support academic advising and help staff provide students with more informed guidance regarding their course schedules and academic workload.

1.4 Environment

The proposed system will be a web application, so users will be able to access it through a standard web browser without needing any additional hardware or software. The application will be hosted on the CAS servers, or through a cloud platform. The final hosting option will depend on CAS requirements, particularly around data privacy, security, and long-term maintenance.

The system is intended to replace the shared calendar that CAS currently maintains manually, while also adding features that help with scheduling and course planning. The main source of information will be CAS course outlines, which are typically available as PDF or Word documents. Other information, such as Mosaic timetables and important academic calendar dates like exam periods, will also be used. Once the information is extracted, it will be stored in a central database that CAS administrators can review, correct, and export.

The scheduling component will use the CP-SAT solver from Google OR-Tools to generate possible assessment dates based on the system's constraints. An LLM will be used to provide short explanations for these scheduling suggestions. During development, the team plans to use a hosted LLM API and will only send publicly available course outline information to it. Since the system may eventually handle confidential information, the team will also explore hosting the LLM directly on CAS servers. The LLM will be kept as a separate part of the system so that the hosted model can later be replaced with a locally hosted one without requiring major changes to the rest of the application.

2 Goals

The project goals include:

1. **Centralized Assessment Calendar**

   Develop a shared calendar that allows professors and CAS administrators to view midterm and assignment deadlines across all undergraduate Software Engineering courses, with filters for course, year, and program. Success will be measured by successfully displaying at least 95% of assessment dates from the course outlines in the test dataset and correctly filtering assessments by course and year.

2. **Automated Course Outline Data Extraction**

   Automatically extract assessment dates, weights, and learning outcomes from course outlines and store them in a centralized CAS database for administrators to review, correct, and export. The system will flag missing, unclear, or incomplete information for human verification. Success will be measured by achieving at least 90% extraction accuracy for assessment dates, weights, and learning outcomes, and correctly flagging at least 90% of predefined missing or incomplete fields across these data types in the test dataset.

3. **Assessment Workload and Conflict Detection**

   Identify weeks with high concentrations of assessments for students in the same academic year and program (undergraduate Software Engineering). Workload clustering will be evaluated based on assessment frequency, timing, type, and weighting. Scheduling conflicts will include multiple assessments scheduled on the same day for the same student cohort, assessments during restricted examination periods, and violations of relevant scheduling policies. Success will be measured by correctly identifying at least 90% of predefined scheduling conflicts in a test dataset and associating all identified conflicts with the correct student cohort.

4. **Scheduling Recommendations**

Provide professors with alternative assessment dates that aim to reduce workload clustering and scheduling conflicts. Recommendations will consider factors such as the number, timing, type, and weighting of assessments within a student cohort to minimize periods of excessive workload. Each recommendation will include an explanation of its reasoning and expected impact, while professors retain full control over whether to accept the suggested changes. Success will be measured by generating at least two feasible alternative dates for 90% of test cases with scheduling conflicts, with at least one alternative reducing the number of identified conflicts in each applicable case.

5. **Automated Accreditation and Policy Compliance Checking**

   Evaluate course outlines against McMaster's course outline policies and applicable CEAB accreditation requirements, identifying missing information and potential compliance gaps. Notify professors and administrators of issues requiring attention. Success will be measured by correctly identifying at least 90% of predefined compliance issues in a test dataset and providing an actionable notification for each detected issue.

3 Stretch Goals

- **Automatic Mosaic integration.** Instead of entering timetable and enrollment data manually, the system could retrieve this information directly from Mosaic. This would help identify student cohorts more accurately based on the courses they are actually taking together.
- **MSAF trend analysis.** If historical MSAF data is made available by the department, the system could use it to identify weeks where students have experienced higher levels of assessment pressure in the past. This information could then be used to improve scheduling suggestions.
- **Compliance checking during outline creation.** Professors could check draft course outlines against McMaster's course outline policy and CEAB requirements before submitting them. This would help identify missing or incorrect information earlier in the process.
- **Student-facing view.** A read-only calendar could be provided for students and academic advisors to view assessment deadlines across multiple courses in one place.

4 Custom Documents (CDs)

The following two Custom Documents were selected to guide our design process throughout the year and support end-user adoption:

1. **Fall Term CD:** Wireframe Report (WIREFRAME)

   The proposed system centers on a visual administrative dashboard featuring a shared assessment calendar, course filtering tools, and scheduling recommendation displays for instructors and CAS administrators. Developing user interface mockups during the Fall term will allow us to define and visualize the UI component interactions and workflow navigation before beginning front-end development. This will also ensure that we are aligning our design with the needs of the users, as well as receiving early feedback on the intuitiveness and usability of the system interface from our clients and stakeholders.

2. **Winter Term CD:** User Manual (USERMAN)

   To ensure a smooth and seamless adoption process by instructors and the CAS administrative staff, a comprehensive user manual will be created to provide clear, step-by-step guidance on operating the software. The manual will cover uploading course outlines, filtering the centralized assessment calendar, exporting database records, and reviewing automated policy compliance flags and scheduling recommendations.

Appendix — Reflection

What went well while writing this deliverable?

Before writing this deliverable, our team created an outline of all the sections we needed to complete, and we divided the parts amongst ourselves based on what we felt confident in writing, as well as what interested us. One of our biggest strengths as a group was our familiarity with each other; one of the first things we set up as a capstone team was a group chat, where we were consistently available to brainstorm and discuss project ideas. Currently, we have not faced any major disagreements, and this has fortunately continued when we were writing the problem statement and goals. One positive aspect to highlight were the reviews for the pull requests. While it would have been easier to give immediate approvals, the members of the team have been carefully cross-referencing the work with the rubric, and adding suggestions to improve the quality of the sections.

What pain points did you experience during this deliverable, and how did you resolve them?

During this deliverable, one of the major pain points we faced as a team was describing the problem and its details. For our project (CAS Midterm and Assignment Optimization), we were not able to find adequate time to meet with our supervisors to further discuss what they were looking to get out of the project. The only information we had to go off of was the information session for this project, as well as the interview we conducted to pitch ourselves and our interest. Fortunately, we took notes during both meetings, and used what we had to gain a solid understanding of the problem. Additionally, given our experience with overwhelming coursework and consecutive midterms (which some of us are currently suffering from), we were able to extrapolate from our personal experiences to better understand the nature of the problem and what we aimed to achieve.

Another pain point we experienced was understanding the structure of the deliverable and how well our content organization was. Aside from the checklist and rubric, we were unsure how well-written and formal our deliverable was, but to address this gap, we decided to bring it up during our initial meeting with our TA advisor, Lucas Dutton. After meeting with Lucas, we asked him questions we had about the deliverable, and thanks to his feedback and clarifications, we were able to write the deliverable with more confidence and understanding.

How did you and your team adjust the scope of your goals to ensure they are suitable for a Capstone project?

When we were speaking with our supervisors, we learned that the project was initially part of a larger initiative to improve course scheduling and workload

management across the CAS department (for both graduate and undergraduate programs). However, the scope proved to be too large, and as a result this project was greatly scoped down to undergraduate courses in the Software Engineering program. Regardless, when our team was first introduced to the project, we discussed whether the project itself was feasible to complete within the capstone project timeframe. After further discussion with the supervisors, we agreed that the project scope was indeed within our capabilities, but that we would have to be careful not to get caught up in the various features we can tackle and prioritize developing and delivering a completed functional project that satisfies the core requirements of the stakeholders rather than just bells and whistles.